



Plan de Gestión de Eventos

Monitoreo, alertamiento y respuesta a incidentes bajo el marco **ITIL v4**,
aplicado a la plataforma **Serena Seguros**

INTEGRANTES

Alatta Torres, Luis Fernando
Canchari Mamani, Liz Bayoleth
Lupaca Mendoza, Marlon Diego

 **MBA Mg. Ing. Rene Alonso Nieto Valencia**

Ciclo 09 • Facultad de Ingeniería
Arequipa, Perú — 2026

🎯 CONTEXTO

¿Por qué un Plan de Gestión de Eventos?

Serena Seguros opera un portal web crítico sobre microservicios en **Docker**. Cada falla o degradación debe detectarse y resolverse antes de impactar al usuario final.

- 🎯 **Objetivo:** estandarizar la detección, clasificación y respuesta ante eventos de infraestructura.
- 🗉 **Alcance:** backend Spring Boot, contenedores Docker y base de datos MySQL (Máster/Réplica).
- 🔄 **Enfoque:** monitoreo proactivo con métricas + gestión reactiva de tickets vía Jira JSM.

📦 STACK MONITOREADO

- 🖥️ Backend Spring Boot (puerto 8080)
- 🚢 Contenedores Docker
- 💾 MySQL Máster / Réplica
- 📈 Prometheus + Grafana
- 📅 Jira Service Management

Tres niveles de impacto en el negocio

✓ INFORMATIVO

CRITERIO

Operación normal: backup OK,
login exitoso, CPU < 65%

ACCIÓN

Ninguna. Se registra en
Prometheus para análisis histórico

⚠ ADVERTENCIA

CRITERIO

CPU > 80%, Disco > 85%, RAM >
90%

ACCIÓN

Monitoreo proactivo. DevOps
revisa sin interrumpir el servicio

! EXCEPCIÓN

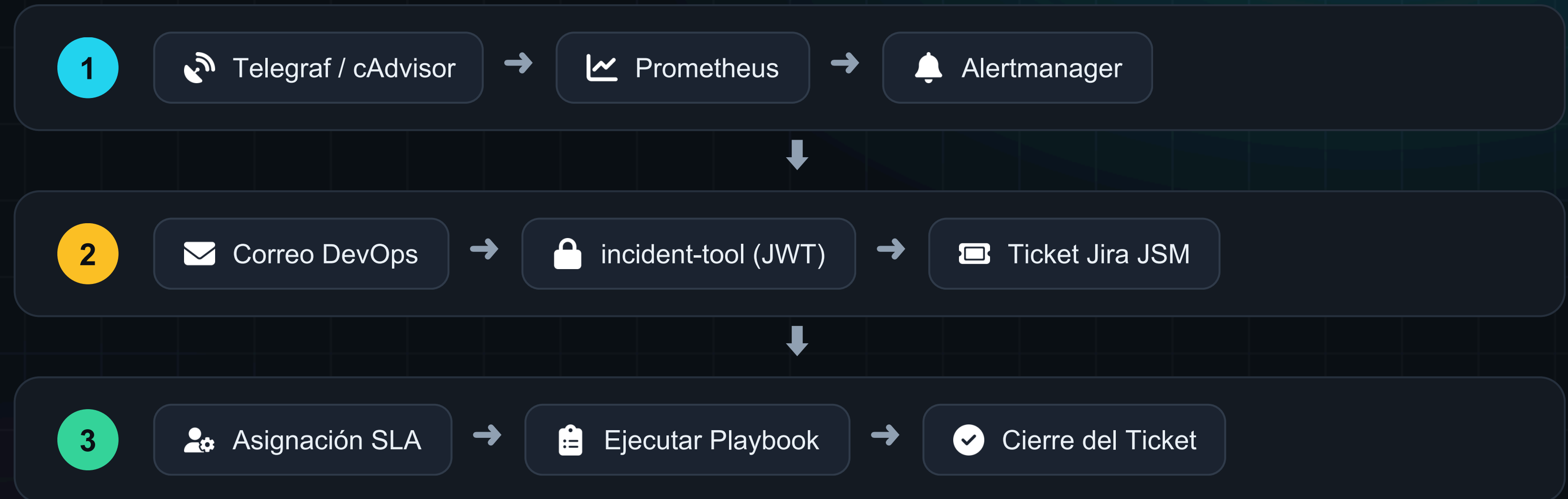
CRITERIO

Backend caído, RAM > 95%,
réplica caída

ACCIÓN

Respuesta inmediata: alerta por
email + ticket en Jira JSM

De la métrica a la resolución del ticket



Dos caminos para detectar una falla



Proactiva — Telemetría

Telegraf y cAdvisor envían métricas de rendimiento a Prometheus de forma continua, sin intervención humana.

DISPARADOR

Falla consecutiva > 30s en una métrica crítica (ej: puerto TCP 8080 de Spring Boot) → incidente automático



Reactiva — Reportes

Reportes generados manualmente por usuarios finales o personal de soporte técnico ante una falla percibida.

DISPARADOR

Portal de autogestión o bandeja de correo enlazada a Jira Service Management

📊 MATRIZ DE PRIORIZACIÓN

Impacto × Urgencia → Prioridad

Urgencia ↑↓ / Impacto ⇌		● Alto	● Medio	● Bajo
⚡ Alta	P1 Crítico	P2 Alto	P3 Medio	
📉 Media	P2 Alto	P3 Medio	P3 Medio	
🌙 Baja	P3 Medio	P3 Medio	P3 Medio	

● P1 · Operación detenida / portal caído

● P2 · Degradación severa del servicio

● P3 · Supervisión preventiva

Tiempos máximos de respuesta y resolución

P1 — Crítico

RESPUESTA	15 min
RESOLUCIÓN	2 horas

Ej: backend caído, RAM > 95% con riesgo de bloqueo general

P2 — Alto

RESPUESTA	30 min
RESOLUCIÓN	4 horas

Ej: CPU al 83% sostenido, almacenamiento > 85%

P3 — Medio

RESPUESTA	2 horas
RESOLUCIÓN	8 horas

Ej: CPU al 68%, retraso menor en base de datos

Reglas que sostienen la operación



Monitoreo Activo 24/7

Monitoreo ininterrumpido en todos los contenedores productivos, sin ventanas ciegas.



Agrupamiento de Alertas

Alertmanager agrupa alertas similares del mismo servicio para evitar la "fatiga de alertas" en la bandeja de entrada.



Regla de Silencios

Durante ventanas de mantenimiento programadas, las alertas se silencian en Prometheus para evitar falsos incidentes y tickets basura en Jira.

Spring Boot fuera de servicio

1. Detección

- Prometheus reporta la métrica de disponibilidad en 0 para `serena-backend`



2. Acción inmediata

- Ingresar al servidor por SSH
- Verificar con `docker ps`
- Si está detenido: `docker compose start backend`
- Revisar logs: `docker logs --tail 100`



3. Cierre

- Verificar que `/api/v1/publico/productos` responda
- Cerrar el ticket en Jira JSM

Almacenamiento superando el umbral

1. Detección

- Alerta de almacenamiento superando el 85% de capacidad



2. Acción inmediata

- Liberar imágenes y volúmenes en desuso:
`docker system prune -a --volumes -f`
- Depurar archivos .log antiguos en
`/var/lib/docker/containers/`



3. Cierre

- Validar con `df -h` que el uso bajó a menos del 70%

 HERRAMIENTAS RECOMENDADAS

El stack de observabilidad



COLECCIÓN DE DATOS

Telegraf

Métricas de sistema, MySQL
y contenedores Docker



BASE DE DATOS

Prometheus

Almacenamiento de series
temporales de alta velocidad



VISUALIZACIÓN

Grafana

Dashboards de CPU, RAM,
transacciones y réplicas



MESA DE SERVICIOS

Jira Service Mgmt

Incidentes, SLAs, flujos de
trabajo e historial

De reactivo a autosuficiente

1

Remediación automatizada

Scripts que ejecuten la limpieza de Docker cuando la alerta de almacenamiento pase a estado firing

2

Monitoreo de replicación

Telegraf debe alertar sobre el estado de sincronización entre MySQL Máster y Réplica

3

Simulacros ("Game Days")

Usar el panel incident-tool para simular caídas controladas y evaluar el tiempo real de reacción

4

Tuning de alertas

Revisar trimestralmente las reglas para evitar falsas alertas que insensibilicen al equipo



Preguntas sobre el Plan de Gestión de Eventos — Serena Seguros



Alatta Torres, Luis
Fernando



Canchari Mamani, Liz
Bayoleth



Lupaca Mendoza, Marlon
Diego

Curso Integrador II: Sistemas · Ciclo 09 · Facultad de Ingeniería
Docente: MBA Mg. Ing. Rene Alonso Nieto Valencia — Arequipa, Perú 2026